

Fast and Efficient Approximate Nearest Neighbor Search for High-Dimensional LLM Embeddings

Nico Hezel¹ , Kai Uwe Barthel^{1,2} , Bruno Schilling¹ ,
Konstantin Schall^{1,2} , Andre Moelle² , and Klaus Jung¹ 

¹ Visual Computing Group, HTW Berlin, Germany www.visual-computing.com
✉ nico.hezel@htw-berlin.de

² vviinn, Berlin, Germany www.vviinn.com

Abstract. The annual SISAP Indexing Challenge benchmarks Approximate Nearest Neighbor Search (ANNS) algorithms under rigorous constraints. This paper presents our submissions for the 2026 edition, addressing both k -Nearest Neighbor Graph (kNNG) construction on 1024-dimensional BGE-M3 embeddings (Task 1), and Maximum Inner Product Search (MIPS) on unnormalized Llama-3.2-8B features (Task 2). Our approach achieved 1st place in Task 2 and 3rd place in Task 1.

To optimize construction speed, we utilize Equi-Voronoi Polytopes (EVP) for efficient quantization, supplemented by targeted reranking strategies to maintain high recall. For MIPS, we transform the asymmetric inner product problem into a Euclidean search space via dimensionality augmentation. To reduce query latency and optimize memory access, we introduce a 1D presorting mechanism via Fast Linear Assignment Sorting (FLAS) prior to graph construction. This significantly improves spatial locality and cache hit rates during subsequent graph traversal.

Source Code: <https://github.com/Visual-Computing/sisap26-deglib>

Keywords: Approximate Nearest Neighbor Search · Exploration Graph

1 Introduction

Nearest neighbor search is a cornerstone of modern search systems, recommender engines, and Large Language Models (LLMs). Items are compared using metrics on high-dimensional feature vectors. The computational effort depends on the distance function, dimensionality, and data type. For massive datasets or complex distance functions, exact comparisons become too slow. Therefore, approximate nearest neighbor search (ANNS) algorithms are indispensable.

The annual SISAP Indexing Challenge benchmarks ANNS approaches under strict constraints. The 2026 edition introduces larger datasets and unnormalized feature vectors. Submissions run in Docker containers restricted to 8 vCPUs and 24 GB of RAM on an AMD EPYC 7F72 processor. The challenge comprises three tasks, of which we address the first two. To ensure evaluation consistency, our experiments were conducted on an Intel Xeon 8581C virtual machine configured with identical resource limits and restricted to AVX2 instructions.

Accepted for publication at SISAP 2026 (Springer LNCS). This pre-print is not the Version of Record and does not reflect post-acceptance improvements or corrections.

Task 1 requires computing an *approximate k -Nearest Neighbor Graph* (kNNG) for $k = 15$. The dataset contains 6.4 million normalized, 1024-dimensional vectors produced from a BGE-M3 embedding model. Similarity is measured via dot product. The goal is to reduce the total computation time, which includes preprocessing, index building, search, and post-processing. Submissions must achieve at least 0.8 average recall within an 8-hour time limit.

Task 2 evaluates *Maximum Inner Product Search* (MIPS) on LLM-inspired workloads under distribution shifts. The dataset consists of 256,000 unnormalized, 128-dimensional embeddings from a Llama-3.2-8B model. The objective is to find the $k = 30$ maximum inner products for a provided set of queries. Unlike Task 1, the evaluation measures only the search phase time. Submissions are limited to 1 hour of runtime and must reach a recall of 0.8.

The 2025 challenge already featured a k -NN graph construction task on 3 million vectors with 384 dimensions. It was won by team *hforest* [6], while refinement-based graph approaches by *BrownCICESE* [4] proved highly competitive. However, the 2026 tasks raise the bar significantly. Task 1 more than doubles the dataset size to 6.4 million vectors and increases the dimensionality to 1024. This massive scale heavily penalizes slow index construction times. Meanwhile, Task 2 focuses on unnormalized LLM embeddings and a distribution shift between dataset and query vectors.

2 Related Work

Exact k -Nearest Neighbor Graphs (kNNG) [8] connect each vertex to its true closest neighbors but are computationally prohibitive to construct. Therefore, approximate structures like kGraph [3] are used. Dong et al. introduced the *graph quality* metric to explicitly measure the overlap between these approximated edges and the exact nearest neighbors. However, directly building an index that inherently possesses high graph quality is still slow. Often, a faster strategy is to quickly build a proximity graph, then extract the true top- k neighbors via a local search initialized directly at each target vertex. While the widely used HNSW [7] builds rapidly, its hierarchical design prioritizes long-distance navigation edges for out-of-sample queries. When forcing a local exploration search on its bottom layer, the algorithm easily gets trapped in local minima due to missing connectivity guarantees. In contrast, the Dynamic Exploration Graph (DEG) [5] is explicitly optimized for this task. It operates on a single, strongly connected layer, ensuring highly efficient local exploration. To accelerate graph construction, vector quantization is highly effective; however, traditional methods often require a time-consuming training phase. Connor et al. [2] propose Equi-Voronoi Polytopes (EVP), a training-free approach mapping high-dimensional vectors to convex polytope vertices. The geometric mapping preserves Spearman similarity correlation and processes vectors independently, ensuring a highly parallelizable conversion. The resulting bit-representations enable rapid distance calculations via Single Instruction, Multiple Data (SIMD) bitwise operations.

3 Contribution

In this paper, we address the requirements of the SISAP 2026 Indexing Challenge by optimizing the Dynamic Exploration Graph (DEG) for high-dimensional and LLM-based workloads. Our contributions are summarized as follows:

- **EVP-Accelerated kNNG Construction:** We accelerate high-dimensional proximity graph construction by integrating EVP quantization into the DEG framework. To counteract the resulting precision loss, we design and systematically evaluate hybrid EVP-FP16 configurations to restore target recall.
- **Dimensionality Augmentation for MIPS:** We apply a lifting technique to embed unnormalized embeddings into a $(d + 1)$ -dimensional space, transforming the asymmetric inner product maximization into a symmetric distance minimization problem compatible with standard proximity graphs.
- **Cache-Aware Graph Layout:** We introduce a 1D presorting step to arrange vectors by geometric similarity prior to graph construction. Storing similar vectors in contiguous memory locations maximizes spatial locality, boosting cache hit rates and reducing query latency during graph traversal.

4 Task 1 with EVP and DEG

To minimize the execution time in Task 1, we integrate Equi-Voronoi Polytopes (EVP) quantization [2] into the Dynamic Exploration Graph (DEG). EVP maps high-dimensional embeddings to ternary values $\{-1, 0, 1\}$, enforcing a constant number of non-zero elements controlled via a hyperparameter. Even asymmetric distance calculations between unquantized FP16 queries and ternary EVP database vectors are highly efficient, requiring only masked additions and subtractions. However, aggressive quantization introduces a precision trade-off by degrading similarity fidelity. Empirical evaluation demonstrates a failure to achieve target recall when relying exclusively on EVP for graph construction and search. The resulting topology requires a hybrid-precision approach and a dedicated reranking phase to recover accuracy while preserving throughput.

4.1 Evaluated Approaches

To evaluate the trade-offs between construction speed and accuracy, we vary vector representation across three pipeline components: **graph construction**, the **database vector** format, and the **query vector**. Five distinct algorithmic configurations are evaluated (top table in Figure 1):

- **DEG FP16:** Graph constructed, stored, and queried entirely using FP16 vectors. Extracting the 15 nearest neighbors directly from the raw graph edges yields a recall below the 0.8 threshold, while increasing the vertex degree to improve accuracy drastically inflates build times. Therefore, a local exploration search is initiated from each vertex to extract the 15 nearest neighbors without compromising construction speed.

- **DEG EVP:** FP16 vectors are quantized to 2-bit Equi-Voronoi Polytopes (EVP). Both graph construction and the subsequent local exploration search rely entirely on the symmetric EVP metric.
- **DEG EVP Asym:** The initial graph construction matches the symmetric EVP Baseline. During local exploration, original FP16 vectors serve as queries, utilizing an asymmetric distance function against the stored EVP database vectors.
- **DEG EVP + Reranking:** Graph construction operates on EVP vectors. Local exploration retrieves an expanded candidate pool exceeding $k = 15$, which is subsequently reranked using exact FP16 features to determine the final nearest neighbors.
- **DEG EVP $\rightarrow$ FP16 repl.:** The index is initially built using EVP vectors. Post-construction, all internal EVP representations are replaced with their original FP16 counterparts. Local exploration then operates exclusively on these float features.

Approach	Graph Construction	Database Vector	Query Vector	Reranking Strategy	Quant [s]	Build [s]	Explore [s]	Rerank [s]
✕ linear EVP	-	EVP	EVP	no	0.55	-	175.40	-
⊕ linear EVP Asym	-	EVP	FP16	no	0.54	-	857.40	-
● DEG FP16	FP16	FP16	FP16	no	-	8.50	0.64	-
■ DEG EVP	EVP	EVP	EVP	no	0.54	3.72	0.38	-
▲ DEG EVP Asym	EVP	EVP	FP16	no	0.55	3.96	1.00	-
◆ DEG EVP + Reranking	EVP	EVP	EVP	yes	0.56	3.91	0.64	0.39
● DEG EVP $\rightarrow$ FP16 repl.	EVP	FP16	FP16	no	0.55	3.84	1.44	-

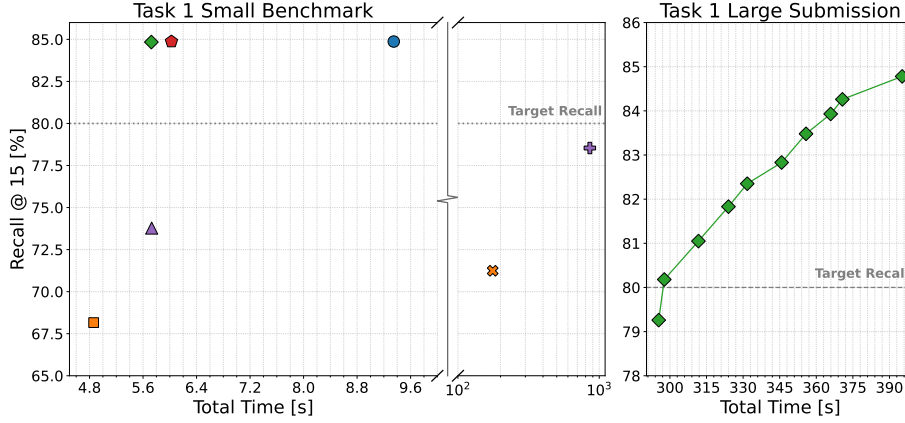


Fig. 1: Task 1 benchmark results. Top: Algorithmic pipeline configurations and detailed timing breakdowns on the small dataset. Bottom-left: Recall@15 vs. total execution time for the small dataset. Bottom-right: Submission Pareto curve for the **DEG EVP + Reranking** approach across hyperparameter configurations on the large 6.4M vector dataset.

4.2 Results

The following results were evaluated on a small provided dataset (about 200,000 vectors) to assess algorithmic efficiency, as shown on the bottom-left of Figure 1. All graphs share identical construction hyperparameters, and exploration parameters are tuned to reach a recall of 0.85 wherever possible.

Graph construction dominates the total execution time across all experiments. For **DEG FP16**, this phase consumes approximately 80% of the total duration. Utilizing EVP vectors halves the construction time but limits the final recall to 0.68 (**DEG EVP**). An exhaustive linear search using only EVP bits shows a hard upper bound of 0.71. Even applying the asymmetric distance function during linear search raises the upper bound to only 0.78. Both upper bounds fall short of the 0.8 target recall. Furthermore, the asymmetric distance computation is several times slower than the symmetric EVP search. This is true for the linear search as well as the exploration time in (**DEG EVP Asym**).

Replacing the asymmetric metric in DEG with a subsequent reranking step (**DEG EVP + Reranking**) yields similar execution times but higher recall rates. For this test, the EVP exploration phase retrieves 50 initial candidates to ensure a high-quality candidate pool for the reranking step. This expanded retrieval pool slightly increases the exploration duration.

Substituting all quantized database vectors with the original FP16 features after graph construction (**DEG EVP** $\rightarrow$ **FP16 repl.**) enables standard graph search without a separate reranking phase. The superior precision of the FP16 metric reduces the required graph traversal depth compared to pure EVP. However, the high computational cost of floating-point distance calculations renders this strategy slightly slower than the **DEG EVP + Reranking** pipeline.

For the large dataset, a hyperparameter search was conducted. The top ten configurations for the **DEG EVP + Reranking** approach are identified and shown in Figure 1 on the bottom-right. Achieving the 0.8 target recall requires a total execution time of 297 seconds. This duration comprises 52% graph construction, 5% EVP quantization, 34% exploration and reranking, and the remaining fraction for disk loading.

5 Task 2 with Dimension Expansion and FLAS

For Task 2, we adapted our graph strategy from Task 1. However, unlike the first task, it evaluates *Maximum Inner Product Search* (MIPS) on unnormalized Llama-3.2-8B embeddings, where varying vector norms distort traditional proximity spaces. Preliminary baseline experiments using an exhaustive linear search with EVP yielded a recall of only 0.08, demonstrating how direction-only quantization strips the magnitude information required for MIPS.

To address these geometric constraints and maximize search throughput, we investigate two independent architectural optimizations:

- **Dimension Augmentation:** A lifting technique inspired by the ALSH framework [9] maps unnormalized vectors into a higher-dimensional space. This transformation stabilizes vector norms, enabling evaluation via both Inner Product and Euclidean distance metrics.
- **Cache-Aware Layout Optimization:** To maximize hardware throughput during the search phase, we apply a spatial reordering strategy via the FLAS algorithm [1] prior to graph construction. This optimizes cache line utilization during traversal and is evaluated on both the original and augmented feature spaces.

5.1 Transforming Inner Products into Euclidean Distances

Maximum Inner Product Search (MIPS) can be reduced to Euclidean Nearest Neighbor Search (NNS) via dimensionality augmentation. For each d -dimensional vector $\mathbf{v}$, we define $\tilde{\mathbf{v}} = [\mathbf{v}^\top, \sqrt{M^2 - \|\mathbf{v}\|^2}]^\top$, where $M \geq \max \|\mathbf{v}\|$. The query is augmented as $\tilde{\mathbf{q}} = [\mathbf{q}^\top, 0]^\top$. The squared Euclidean distance satisfies:

$$\|\tilde{\mathbf{q}} - \tilde{\mathbf{v}}\|^2 = \|\mathbf{q}\|^2 + M^2 - 2\mathbf{q}^\top \mathbf{v} \quad (1)$$

Since $\|\mathbf{q}\|^2$ and M^2 are constant for a fixed query, minimizing $\|\tilde{\mathbf{q}} - \tilde{\mathbf{v}}\|^2$ is equivalent to maximizing $\mathbf{q}^\top \mathbf{v}$. Geometrically, this maps vectors onto a $(d+1)$ -dimensional hypersphere of radius M , rendering the problem directly compatible with standard Euclidean proximity graphs. Figure 2 shows an example of this MIPS reduction in 2D.

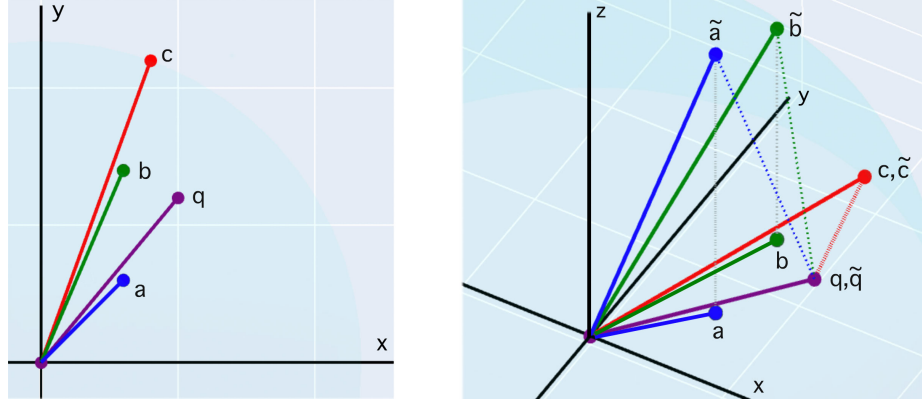


Fig. 2: Geometric embedding of MIPS. Left: In 2D, $\mathbf{c}$ is the vector maximizing the inner product with $\mathbf{q}$. Right: The lifted vectors $\tilde{\mathbf{a}}$, $\tilde{\mathbf{b}}$, and $\tilde{\mathbf{c}}$ reside on a 3D sphere, where minimizing the Euclidean distance to the augmented query $\tilde{\mathbf{q}}$ recovers the maximum inner product.

5.2 Cache-Aware Graph Construction via 1D Presorting

While the Dynamic Exploration Graph efficiently handles evolving datasets, its default construction phase processes vectors in an arbitrary insertion order. To optimize the index for modern memory hierarchies in static workloads, we introduce a presorting step prior to vertex insertion, utilizing the Fast Linear Assignment Sorting (FLAS)³ algorithm [1] to sort all vectors one-dimensionally by their similarity.

This strategy maps geometrically similar vectors onto contiguous physical memory addresses. Constructing the DEG from this presorted sequence ensures local neighborhoods align with consecutive cache lines. Consequently, the resulting spatial locality improves CPU cache hit rates and mitigates high-cost main memory fetches, significantly reducing overall query latency.

5.3 Results and Evaluation

The evaluation for Task 2 focuses on the trade-off between search latency and recall. Figure 3 illustrates the performance of the baseline graph against our layout and dimensionality optimization strategies.

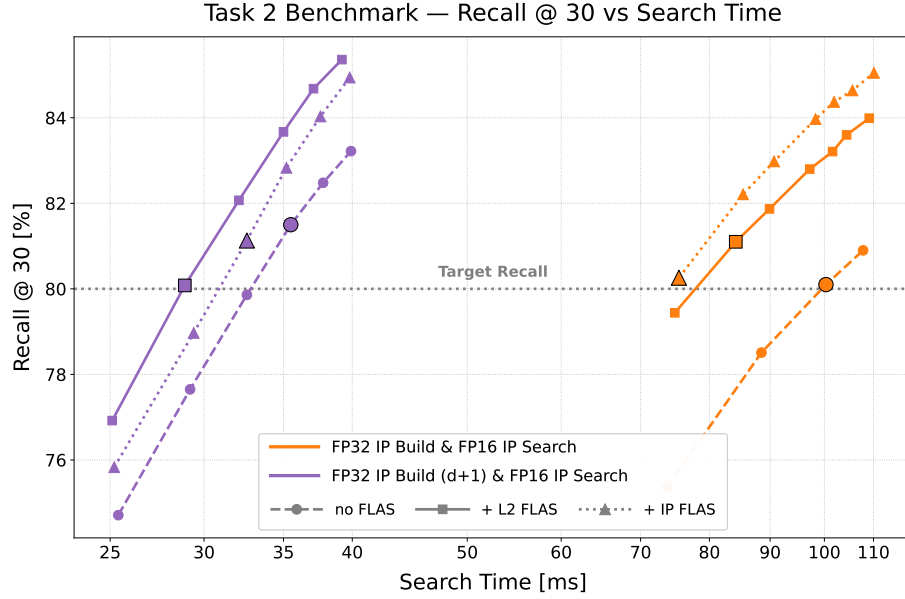


Fig. 3: Task 2 query latency versus recall, comparing the baseline MIPS against FLAS sorting types on the right and dimension expansion + FLAS approaches on the left.

³ https://github.com/Visual-Computing/LAS_FLAS/

The baseline Dynamic Exploration Graph, constructed in the original unnormalized FP32 space and searched via the Inner Product metric using FP16 features, requires approximately 100 ms to complete all queries due to the distorted geometric space. Integrating the FLAS sorting step prior to graph construction reduces query latency to 80 ms, achieving a 20% performance gain solely through enhanced spatial locality. Within this unnormalized space, configuring FLAS to utilize the Inner Product metric yields superior results compared to an L_2 configuration, as aligning the sorting metric with the underlying search space preserves local neighborhood structures. Applying dimension augmentation without presorting slashes search time to 33 ms, a 67% improvement over the baseline. This reduction demonstrates how lifting embeddings into the $(d+1)$ -dimensional space stabilizes vector norms and resolves the geometric discrepancies of unnormalized MIPS. Crucially, within this augmented space, the optimal metric for FLAS shifts to L_2 . Because the wide dynamic range of Inner Product values introduces severe quantization inaccuracies during the FLAS sorting phase, employing the bounded L_2 metric post-extension mitigates these precision errors.

Compounding both methodologies delivers the highest performance. Combining dimension extension with the L_2 -configured FLAS reordering reduces query latency to 29 ms, achieving an additional 12% reduction over the unsorted, lifted approach.

6 Conclusion

In this paper, we presented our contributions to the SISAP 2026 Indexing Challenge by optimizing the Dynamic Exploration Graph (DEG) for high-dimensional, distribution-shifted LLM workloads. For Task 1 (normalized embeddings), we utilized Equi-Voronoi Polytopes (EVP) quantization to accelerate graph construction. Supplemented by an FP16 reranking step, this approach reduced total execution time on the small dataset from 9.35 s to 5.72 s, a 39% performance improvement, while maintaining target recall. For Task 2 (unnormalized MIPS), we resolved geometric discrepancies by lifting the feature dimensions, cutting baseline query latency from 100 ms to 33 ms. Compounding this transformation with a spatial data reordering step via the FLAS algorithm further reduced latency to 29 ms by maximizing CPU cache utilization. Our results demonstrate the effectiveness of decoupling geometric transformations and cache-aware memory layouts from the baseline graph topology to optimize performance trade-offs.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Barthel, K.U., Hezel, N., Jung, K., Schall, K.: Improved evaluation and generation of grid layouts using distance preservation quality and linear assignment sorting. *Computer Graphics Forum* **42**(1), 261–276 (2023)
2. Connor, R., Dearle, A., Claydon, B.: Equi-Voronoi Polytopes: A Geometric Basis for 2-bit Quantisation. In: Amato, G., Mic, V., Traina, A.J.M., Messina, N., Amsaleg, L., Guðmundsson, G.P., Jónsson, B.P., Vadicamo, L. (eds.) *SISAP. Lecture Notes in Computer Science*, vol. 16134, pp. 288–302. Springer (2025)
3. Dong, W., Charikar, M., Li, K.: Efficient k-nearest neighbor graph construction for generic similarity measures. In: Srinivasan, S., Ramamritham, K., Kumar, A., Ravindra, M.P., Bertino, E., Kumar, R. (eds.) *Proceedings of the 20th International Conference on World Wide Web*. pp. 577–586. ACM, New York, NY, USA (2011)
4. Foster, C., Magdaleno-Gatica, S., Kimia, B.: Refinement-Based Graph Construction for Search in Low-Memory Systems. In: Amato, G., Mic, V., Traina, A.J.M., Messina, N., Amsaleg, L., Guðmundsson, G.P., Jónsson, B.P., Vadicamo, L. (eds.) *SISAP. Lecture Notes in Computer Science*, vol. 16134, pp. 450–458. Springer (2025)
5. Hezel, N., Barthel, K.U., Schilling, B., Schall, K., Jung, K.: Dynamic exploration graph: A novel approach for efficient nearest neighbor search in evolving multimedia datasets. In: *MultiMedia Modeling: 31th International Conference, MMM 2025*. pp. 333–347. Springer Nature Singapore (2025)
6. Imamura, Y., Shinohara, T., Higuchi, N., Hirata, K., Kuboyama, T.: Hilbert forest in the sisap 2025 indexing challenge (2025)
7. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **42**(4), 824–836 (2020)
8. Paredes, R., Chávez, E.: Using the k-nearest neighbor graph for proximity searching in metric spaces. In: Consens, M.P., Navarro, G. (eds.) *SPIRE. Lecture Notes in Computer Science*, vol. 3772, pp. 127–138. Springer (2005)
9. Shrivastava, A., Li, P.: Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips). In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N.D., Weinberger, K.Q. (eds.) *NIPS*. pp. 2321–2329 (2014)